# Bangladesh Transport Infrastructure Case

## Assignment 2: Building Components for Data-Driven Simulation

EPA133A: Advanced Simulation

Team 14

Delft University of Technology

TU Delft

# Bangladesh Transport Infrastructure Case

## Assignment 2: Building Components for Data-Driven Simulation

by


Team 14

| | |
|---|---|
| Bayu Jamalullael | 6367984 |
| Brenda Escobar | 6512191 |
| Brian Parsaoran | 6147674 |
| Taufik M. Yusup | 6378056 |
| Zhafran Sidik | 5658411 |


6 March 2026


Faculty of Technology, Policy and Management (TPM)


TU Delft

# Contents


**1 Introduction** **1**
1.1 Context and Motivation . . . 1
1.2 Goal and Scope of This Assignment . . . 1

**2 Data Preparation** **2**
2.1 Dataset Description . . . 2
2.2 Creating Model-Ready CSV . . . 2
2.3 Processing Functions . . . 3
2.3.1 Road Extraction - `select_lrps()` . . . 3
2.3.2 Bridge Deduplication - `get_bridges_lrp_and_fix_double()` . . . 3
2.3.3 Distance Utility - `haversine()` . . . 3
2.3.4 Spatial Matching - `find_nearest_bmms_bridge()` . . . 3
2.3.5 Record Integration - `combined_as_a_road()` . . . 3
2.3.6 Distance Computation - `calculate_distances()` . . . 3

**3 Model Design and Component Architecture** **4**
3.1 Model Overview . . . 4
3.2 Component Design . . . 4
3.2.1 Changes in `components.py` . . . 4
3.2.2 Changes in `model.py` . . . 5
3.2.3 Changes in `model_run.py` . . . 5
3.3 Modularity and Cohesion . . . 5

**4 Results and Discussion** **6**
4.1 Experimental Setup . . . 6
4.2 Results . . . 6
4.3 Bonus Questions . . . 8
4.3.1 Identification of Priority Bridges for Investment . . . 8
4.3.2 Modularity and cohesion . . . 9

**5 Conclusion and Reflection** **10**
5.1 Conclusion . . . 10
5.2 Limitations . . . 10
5.3 Future Improvements . . . 11

**References** **12**

**A Data and Code Availability** **13**

**B Use of AI Tools** **14**
B.1 How AI Was Used . . . 14
B.2 How AI Was Not Used . . . 14

**C Team Contributions** **15**

# 1 Introduction

## 1.1. Context and Motivation

Chittagong Port is the largest and busiest port in Bangladesh, it handles approximately 90 percent of the country's export-import trade. Dhaka is the capital and largest city of Bangladesh, with a population of over 20 million [3]. The Dhaka-Chittagong Highway (N1) is one of, if not the, most critical road route in Bangladesh. As much as 96 percent of the millions of containers handled in Chittagong Port were transported along the N1 highway in FY 2024-25. Although it is the country's main container transport artery, it is not meeting the required standards to due excessive traffic pressure, infrastructure limitations, and poor management [2].

This road, like most roads in Bangladesh, is dependent on bridges, culverts, and other structures designed to manage frequent flooding in the region. The vulnerability to seismic failure of these structures adds an additional layer of risk to transport reliability. Bridges, especially ones in deteriorating conditions, are prone to obstruction, failure, and need for maintenance, resulting in delays. Delays in a major road such as Chittagong-Dhaka can have ripple effects in the economy: supply chains slow, transportation costs rise, and entire industries face disruptions.

So, how to assess which bridges should be priority for infrastructure investment?

## 1.2. Goal and Scope of This Assignment

The goal of this study is to automatically generate an agent-based model using `Mesa` to study the effects of bridge maintenance or unavailability on traffic throughout the most important highway in Bangladesh, the N1.

The model simulates vehicles driving or waiting on the road. Each simulation run, random bridges can break down based on their condition and increase delay time. The model generated provides information on how bridge conditions can propagate delays along the route. This document reports how the modeling process was carried out and the results obtained.

# 2

# Data Preparation

## 2.1. Dataset Description

The primary dataset is `_roads3.csv`, a nation-wide road-survey file containing 51 348 Location Reference Point (LRP) records. Each record carries eight attributes: **road** (identifier), **chainage** (cumulative km), **lrp** (unique code), **lat**/**lon** (WGS 84 coordinates), **gap** (`BS`/`BE` markers for bridge boundaries), **type** (e.g. *KmPost*, *Bridge*, *Culvert*), and **name** (human-readable label).

Bridge metadata comes from `BMMS_overview.xlsx`, exported from the *Bridge Management and Maintenance System* (BMMS) of Bangladesh's Roads and Highways Department. It records each structure's physical length, condition rating (A-D), name, and coordinates. Much of this data was partially entered by human operators, so records may contain positional inaccuracies, dual-carriageway duplicates, or missing fields.

From the full dataset, only the N1 highway (Chittagong-Dhaka) is used: filtering from `LRP235b` to `LRPS` yields 1 342 LRP records that, combined with matched BMMS entries, form the model-ready dataset.

## 2.2. Creating Model-Ready CSV

A six-step pipeline in `clean roads and bridges.ipynb` transforms the two source files into `df_road_file.csv`:

1. **Road extraction** (`select_lrps`). Isolates the N1 LRP range `LRP235b`–`LRPS`, discarding all other roads.
2. **Bridge cleaning** (`get_bridges_lrp_and_fix_double`). Deduplicates BMMS records by keeping only the right-side `(R)` bridge per chainage, since the model simulates the Chittagong-to-Dhaka direction (the right carriageway in left-hand traffic).
3. **Spatial matching** (`find_nearest_bmms_bridge`). Links each road-side bridge or culvert to its nearest BMMS record via Haversine distance (thresholds: 50 m for culverts, 200 m for bridges); unmatched structures default to condition `'B'`.
4. **Record integration** (`combined_as_a_road`). Merges road and bridge data, classifying each row as *source*, *sink*, *link*, or *bridge* with unique IDs from 1 000 000. The originating LRP code is preserved in each row.
5. **Distance computation** (`calculate_distances`). Fills missing lengths using Haversine distances between consecutive entries, preserving BMMS bridge spans and zero-length source/sink entries.
6. **Export**. Writes `df_road_file.csv`.

The output contains 648 rows (1 source, 564 links, 82 bridges, 1 sink). Of the 82 bridges, 26 are rated `A` (good), 53 `B` (fair), 3 `C` (poor), and 0 `D` (very poor).

## 2.3. Processing Functions

### 2.3.1. Road Extraction - `select_lrps()`

Extracts a contiguous LRP stretch from `_roads3.csv`. It accepts a file path, road identifiers, start/end LRP codes, and an optional `start_name` for disambiguation. The function filters by road, locates the bounding indices, slices the inclusive range (reversing if needed for the desired travel direction), and concatenates across roads. A `ValueError` is raised if an LRP is missing or misordered. For our model it produces the 1 342-record N1 segment from `LRP235b` to `LRPS`.

### 2.3.2. Bridge Deduplication - `get_bridges_lrp_and_fix_double()`

Loads BMMS bridge data and removes duplicates in two passes: first by grouping on chainage and retaining the right-side (R) entry, then by removing any surviving left-side (L) records. The right-side bridge is deliberately chosen because the model simulates truck movements in the Chittagong-to-Dhaka direction. In the BMMS dataset, each bridge is recorded twice for each travel direction, with the left-side (L) entry reflecting the bridge encountered when travelling from Dhaka to Chittagong. The right-side (R) entry reflects the bridge as encountered when travelling from Chittagong to Dhaka.

### 2.3.3. Distance Utility - `haversine()`

Computes the great-circle distance in metres ($d$) between two lat/lon points using the Haversine formula:

$$a = \sin^2\!\left(\frac{\Delta\varphi}{2}\right) + \cos\varphi_1\,\cos\varphi_2\,\sin^2\!\left(\frac{\Delta\lambda}{2}\right), \qquad d = 2R\,\cdot \operatorname{atan2}\!\left(\sqrt{a},\,\sqrt{1-a}\right),$$

where $R = 6{,}371{,}000$ m. $\varphi$ represents the latitude and $\lambda$ is the longitude. This spherical approximation is used throughout the pipeline for inter-LRP and LRP-to-bridge distances.

### 2.3.4. Spatial Matching - `find_nearest_bmms_bridge()`

Performs a linear scan over the bridge DataFrame to find the nearest BMMS record to a given coordinate using `haversine()`, returning the closest row and its distance, or `None` and infinity for an empty DataFrame, and is used by the integration step to decide whether real BMMS attributes or placeholder values are assigned. The proximity thresholds are 50 m for culverts and 200 m for bridges, reflecting the physical size and spatial density of each structure type as well as the positional uncertainty in the source data — culverts are small (2–15 m span) and densely spaced, so a tight radius avoids false matches while absorbing the ∼5–15 m GPS error present in both datasets, whereas bridges are much larger (20–500+ m) and spaced farther apart, and their matching uses the midpoint of `BS`/`BE` markers against a BMMS coordinate that may refer to an abutment or centroid, easily introducing 50–150 m of offset on long spans, which a 200 m radius accommodates without confusion.

### 2.3.5. Record Integration - `combined_as_a_road()`

The central function merging road segments and bridge data into a single DataFrame. It iterates via a while-loop, branching by segment type:

- **Source/Sink:** First and last LRPs become origin and destination nodes with zero length.
- **Culvert:** Creates a link, then appends a bridge entry if a BMMS match exists within 50 m.
- **Bridge (BS/BE pairs):** Consumes start/end markers together; searches within 200 m for BMMS data. Unmatched bridges default to condition `'B'`. A zero-length link follows to avoid double-counting.
- **Default:** All other types become plain links.

### 2.3.6. Distance Computation - `calculate_distances()`

Fills missing `length` values by computing Haversine distances between consecutive rows. Pre-set lengths are preserved: zero for sources, BMMS spans for bridges, and zero for post-bridge links. All values are rounded to integers. The result is saved as `df_road_file.csv`, the primary input for the agent-based simulation model. Each row also carries the originating LRP code from the road survey in the `lrp` column.

# 3

# Model Design and Component Architecture

The model used for the simulation was developed from a provided component-based model to represent vehicle movements along a road. Component-based modelling is an approach where a system is build by assembling smaller, self-contained, reusable components. Each component represents a specific part of the system, it encapsulates its own properties and rules for how or if these properties have to change.

In this section, we describe an overview of the model, the modifications made to components in the model, and expand on modularity and cohesion.

## 3.1. Model Overview

The model simulates the transit of goods in trucks through the Bangladeshi road network. For this project, trucks move through the N1 highway from Dhaka to Chittagong. This model uses cleaned data generated through the steps outlined in Chapter 2. The primary structural components of the road model are Source, Link, Bridge, and Sink elements. Trucks are dispatched from a Source every five minutes, traversing a sequence of Bridges, Links (road segments) before exiting at a Sink. The objective of the model is to simulate the effects of bridge maintenance conditions on truck travel time across eight scenarios that reflect the severity of the conditions.

The simulation is done using the `Mesa` package on Python. The simulation lasts 7200 ticks, where one tick represents one minute. Thus, this represents 5*24 hours.

Additionally, the physical characteristics of the model are data-driven, meaning location, length, and condition of bridges and road segments are derived from real-world datasets. This allows an automatic generation of the road model structure.

## 3.2. Component Design

Given that this research builds upon an existing component-based model, the overall system architecture remains unchanged, and efforts focus on modifying specific component functionalities. Changes are primarily on the Bridge component, and some complementary modifications as described below.

### 3.2.1. Changes in `components.py`

This script outlines how the infrastructure and the agent (i.e., trucks) are defined.

The Bridge subclass is expanded to incorporate the bridge condition and the test scenarios. A specific breakdown probability is selected from a matrix (Conditions A to D) using the `scenario` index, where poorer condition ratings correlate with higher probabilities. A bridge is flagged as `is_broken` if a random float falls below this probability. When broken, the `get_delay_time` function calculates a delay time

sampled from distributions that scale with bridge length. This uses a triangular distribution for the longest bridges (>200m) and uniform ranges for shorter ones.

Attributes `total_waiting_time` and `total_vehicle_delayed` are also added to record the total delay accumulated and the number of trucks delayed for a bridge.

Moreover, in the `Vehicle` agent subclass, the speed has been changed from 50 km/h to 48 km/h to better reflect the average speed.

### 3.2.2. Changes in `model.py`

The `model.py` script generates the 'main' model, which is the `BangladeshModel`. Several aspects have been modified in its instantiation. The attribute `self.to_remove=[]` is added, which queues the agents (i.e., the trucks) and only removes them after the data collection is done, as the agents are otherwise removed in the agent step. Moreover, a `roads` parameter has been added with a default value of N1 for this research, while still allowing flexibility should the simulation be done for other highways. A `scenario` parameter was also added to allow for a scenario to be taken in as an argument to test out the different scenarios, which are later elaborated in chapter 4.

Finally, a `DataCollector` is added to capture the agents generated, their generation time, and the time they are removed (`generated_at_step` and `removed_at_step`, respectively). An additional parameter was later added to this function to capture the LRPName (`lrp`), the name of the bridge (`name`), and the total accumulated delay (`total_waiting_time`) for each bridge to complete the bonus task.

In the `generate_model` function, a slight modification is made to the instantiation and identification of the infrastructure step so that the bridge condition and selected scenario can be passed as arguments. This is done by adding `row['condition']` and `self.scenario`.

### 3.2.3. Changes in `model_run.py`

The `model_run.py` script is where the simulation is executed. Here, the run time was adjusted from 1000 ticks to 72000 ticks. To validate the results, the simulation is run per scenario with ten randomly generated seeds. This is done to ensure the same randomised seeds are used throughout all scenarios, ensuring direct comparison of scenarios for each iteration.

An additional variable `roads` was added to enable easier extension of the roads to be used in the simulation. This variable in the list will contain all the roads to be simulated, which, in the current implementation, is set to only `N1`. This variable will be passed as a parameter during model initiation.

A loop was added to enable automated execution of simulations for different scenarios with several iterations with respect to different seeds. This was done to enable full scenario simulation with a single execution of `model_run.py`.

Data regarding Vehicle agents is collected using the `DataCollector` into CSV files for each scenario, containing the data for all ten seeds. Simultaneously, data regarding the total delay accumulated by the bridge is also collected in the same process. This is done to streamline the data collection process while enabling separate processing for different purposes of the data collected.

The main outputs of the simulation are the simulation results for each scenario, which will have information of the duration of vehicles which have completed the trip. This is done by subtracting the (`removed_at_step`) and (`generated_at_step`). All iterations with different seeds will be merged into a single data frame for each scenario, which will be saved as a CSV file.

Additional outputs to be produced with information of total delay accumulated (`total_waiting_time`) for all 82 bridges (represented by `lrp` and `name`), which will be analysed for the bonus task. The results will also be saved in a separate CSV file for each scenario.

## 3.3. Modularity and Cohesion

The redesign of the final code followed an idea to separate responsibilities of experiment control, model behaviour, and structural data, which is guided by modular decomposition principles [1]. It aims to remove the function of the configuration unit in the model by moving scenario loops and seed control to the experiment script and introducing explicit constructor arguments (`scenario`, `roads`).

# 4

# Results and Discussion

## 4.1. Experimental Setup

The simulation will be performed in 8 different scenarios as described in the Table 4.1. Each scenario will be iterated 10 times with differing seeds. The seeds are randomly generated and stored in a list to be used for all scenarios to preserve comparability.

**Table 4.1:** Probability of bridge breakdown per scenario

| Scenario | Cat A % | Cat B % | Cat C % | Cat D % |
|---|---|---|---|---|
| 1 | 0 | 0 | 0 | 5 |
| 2 | 0 | 0 | 0 | 10 |
| 3 | 0 | 0 | 5 | 10 |
| 4 | 0 | 0 | 10 | 20 |
| 5 | 0 | 5 | 10 | 20 |
| 6 | 0 | 10 | 20 | 40 |
| 7 | 5 | 10 | 20 | 40 |
| 8 | 10 | 20 | 40 | 80 |

To automate the simulation for all scenarios, a simple loop is employed to iteratively build the model with differing `seeds` generated previously, run the simulation as defined in the `run_length`, collect the data and append the results to a data frame. The looping index used as the `scenario` index, since it has been designed to be passed as parameter during the model initiation.

No congestion simulated in the model to limit complexity in the design and expected execution time. The output of the simulation will be the information of travel duration time for every vehicles which have completed the full trip.

The simulation was executed on two different machines to observe the reproducibility and the variance of simulation execution time.

## 4.2. Results

Figure 4.1 presents the distribution of mean travel time across 10 replications for each scenario. Under baseline conditions (Scenario 0), the average travel time is approximately 288 minutes, with negligible variability across replications. Scenarios 1 and 2 exhibit identical results, indicating that low breakdown probabilities for lower-quality bridges do not produce measurable system-wide delay effects.

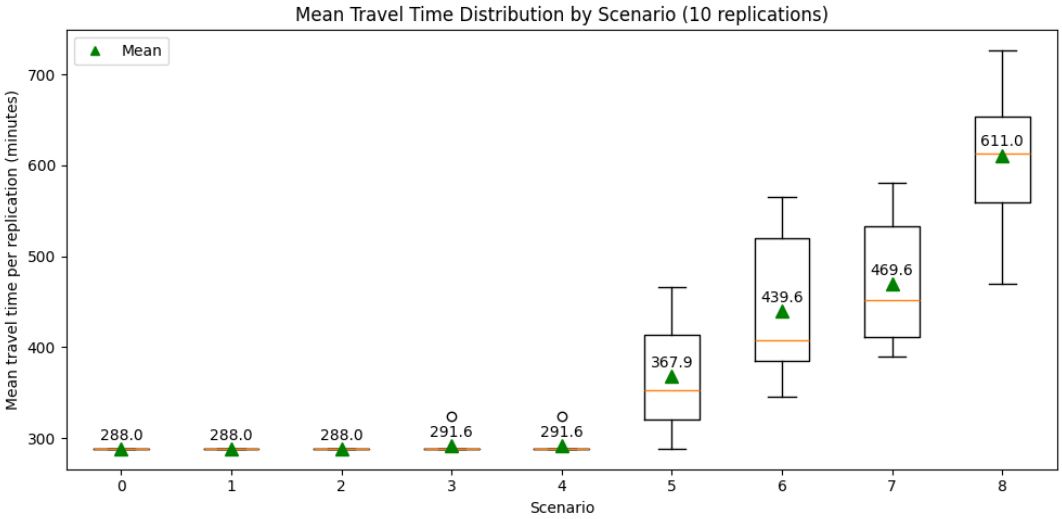


**Figure 4.1:** Distribution of mean travel time across scenarios (10 replications each).

A slight increase in mean travel time is observed in Scenarios 3 and 4. However, the boxplots reveal that this increase is not systematic across replications. The median remains close to the baseline value, and the marginal rise in the mean is primarily driven by isolated outlier realizations. This suggests that while breakdowns begin to occur in these scenarios, their impact is sporadic and does not yet alter overall network performance.

A clear structural shift occurs from Scenario 5 onward. Both the median and mean travel time increase substantially, and the interquartile range widens significantly. Scenario 5 marks the transition from a stable regime to a disruption-dominated regime, where bridge failures consistently affect system performance. As the breakdown probabilities increase further in Scenarios 6–8, travel time rises nonlinearly, reaching approximately 611 minutes in Scenario 8.

The widening spread in higher scenarios indicates that increasing bridge breakdown probabilities not only raise expected travel time but also amplify stochastic variability. In other words, higher disruption levels introduce both greater delay and greater uncertainty in travel outcomes. This behavior is consistent with the increasing breakdown probabilities shown in Table 4.1, particularly for Category C and D bridges.

## 4.3. Bonus Questions

In this section, the two bonus questions in this assignment is discussed.

### 4.3.1. Identification of Priority Bridges for Investment

To determine which five bridges on the N1 corridor should be prioritized for investment, Scenarios 1–7 were weighted equally. The objective was to identify bridges that contribute most significantly to system-wide delay under varying disruption levels.

This selection is based on two main assumptions:

- Delay increases with both the probability of bridge breakdown and the structural characteristics of the bridge, particularly its length.
- Investment is assumed to improve bridge condition, thereby reducing breakdown probability and associated delay.

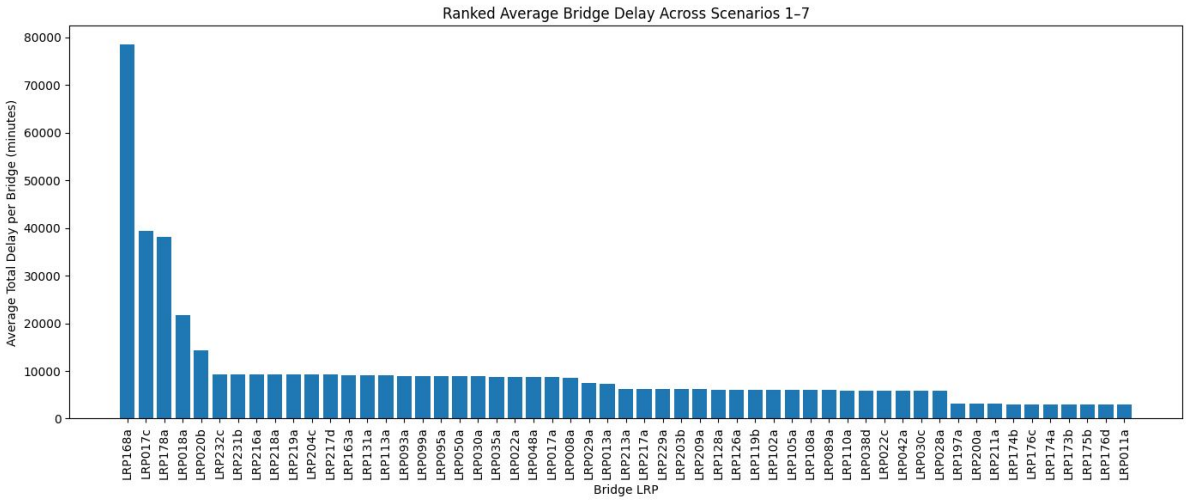


**Figure 4.2:** Ranked average bridge delay across Scenarios 1–7.

Figure 4.2 presents the ranked average bridge delay across Scenarios 1–7. The distribution is highly skewed, with a small subset of bridges contributing disproportionately to total delay. Five bridges clearly stand out as dominant contributors.

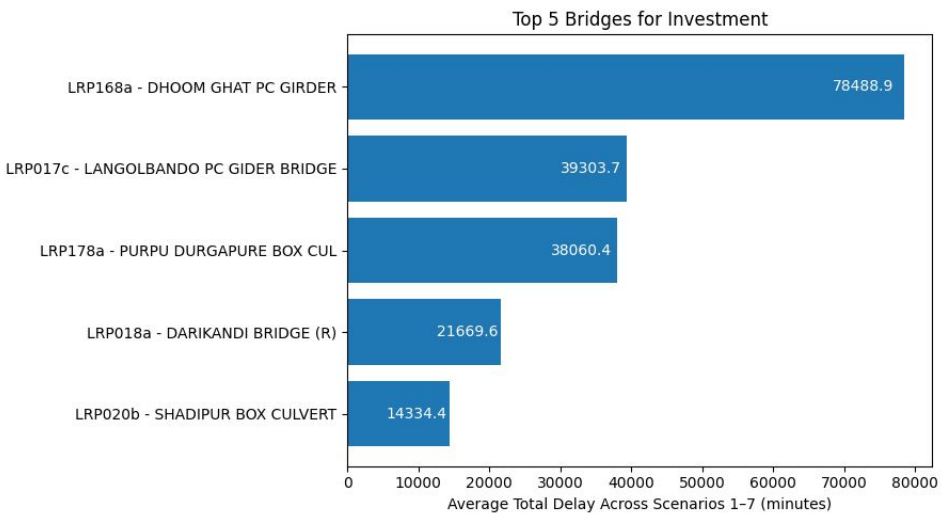


**Figure 4.3:** Top five bridges ranked by average delay contribution (Scenarios 1–7).

Figure 4.3 shows the five bridges selected for investment based on their average delay contribution.

Interestingly, while the two highest-ranked bridges are among the longest structures in the network, not all selected bridges are long. As shown in Figure 4.4, some relatively short bridges also exhibit substantial delay contributions.

| id | model_type | name | lat | lon | length | condition |
|---|---|---|---|---|---|---|
| 1000236 | bridge | DHOOM GHAT PC GIRDER | 22.915667 | 91.525944 | 221 | B |
| 1000598 | bridge | LANGOLBANDO PC GIDER BRIDGE | 23.654986 | 90.575902 | 160 | B |
| 1000594 | bridge | DARIKANDI BRIDGE (R) | 23.650889 | 90.582638 | 20 | B |
| 1000197 | bridge | PURPU DURGAPURE BOX CUL | 22.823944 | 91.552833 | 12 | C |
| 1000586 | bridge | SHADIPUR BOX CULVERT | 23.634111 | 90.597027 | 10 | B |

**Figure 4.4:** Characteristics of the top five bridges based on simulation (length and condition).

| id | model_type | name | lat | lon | length | condition |
|---|---|---|---|---|---|---|
| 1000236 | bridge | DHOOM GHAT PC GIRDER | 22.915667 | 91.525944 | 221 | B |
| 1000598 | bridge | LANGOLBANDO PC GIDER BRIDGE | 23.654986 | 90.575902 | 160 | B |
| 1000256 | bridge | LAMUA BRIDGE | 22.962417 | 91.441694 | 94 | B |
| 1000546 | bridge | Bhaber char Bridge(R) | 23.556444 | 90.658861 | 30 | A |
| 1000558 | bridge | Anarpara Bridge (R) | 23.572166 | 90.645111 | 30 | A |

**Figure 4.5:** Characteristics of the top five longest bridges.

If we compare the top five bridges with the highest delay time against the longest bridges, as shown in Figure 4.5, we can observe that delay contribution is not solely determined by bridge length. Network position and traffic exposure likely play an important role in amplifying the system-wide impact of certain bridges. In other words, some shorter bridges may lie on critical segments of the corridor where breakdowns disrupt a large volume of vehicles.

The results therefore suggest that targeted investment should prioritise bridges with the highest system-level delay contribution rather than relying solely on structural size or category classification.

### 4.3.2. Modularity and cohesion

Drawing on decomposition principles discussed by Hofmann [1], the model is redesigned to strengthen modularity by restructuring responsibilities across layers. The final design explicitly separates data (CSV input), model logic (agents and scheduler), and experiment control (scenario and seed loops). The experiment script now manages scenarios and repetitions externally, while the model focuses solely on system behaviour. This separation aims to reduce interdependence between configuration, execution, and behavioural logic.

The redesign also improves cohesion by relocating variability into parameters. Scenario and condition are passed during model and agent initialisation, embedding breakdown probability as a parameter rather than a fixed assumption. By parameterising breakdown probabilities and passing them explicitly through the model constructor, the code working with controlled inputs rather than implicit behaviours. This aligns with modular design principles that aim to confine change to localised components.

Furthermore, the final code restructures extensibility by allowing the list of roads to be passed as a model argument. Instead of hardcoding the road selection inside the model generation process, the model now receives structural configuration externally. This design choice enables easier selection of roads without modifying internal functions.

# 5

# Conclusion and Reflection

## 5.1. Conclusion

This study set out to automatically generate a Mesa model to simulate vehicle movements and delay time along a major road in Bangladesh (N1), where delay time was formulated as a function of length and bridge breakdown status. The primary objective was to locate the bridges that generated the highest waiting time across multiple scenarios in order to identify where investment should be prioritized. Based on the results obtained, the modeling goal was achieved.

The use of a component-based model as starting point significantly simplified the model development process. Instead of designing a Mesa model from scratch, efforts were concentrated on modifying parameters and functionalities within the bridge component, implementing complementary adjustments, and preparing input data in a format suitable for the model. While future improvements can be made to the algorithm, this modular structure made the overall process more efficient and allowed the adaptability of the simulation to different configurations.

The top five bridges identified represent priority candidates for targeted investment for improvements, as interventions at these points are likely to yield the greatest reduction in overall truck delay.

## 5.2. Limitations

This section describes some limitations the model has in terms of how realistic and useful the outputs are, and on efficiency of use for further use and exploration.

Assuming a bridge's probability to be unavailable or need maintenance just based on its condition sidelines other factors such as type of structure, length, and relative vulnerability to geographic factors such as flooding. And while the physical attributes of the road are based on real data, the movement behavior of vehicles was significantly simplified for exploration and does not necessarily reflect how or how many vehicles move through the actual road.

Accounting for additional factors or building more complex behavior functionalities was outside the scope of this stage of the research. Yet, we mention them to reflect on the assumptions that led to these results.

The current implementation imposes a limitation on scaling the simulation. Given the algorithm's structure, the model took considerable runtime to execute across all scenarios and seeds. The simulation for 1 scenario has an average duration of 17 minutes (totalling around 2 hours 15 minutes across all scenarios) on a computer with typical specifications (8-core processor, 16GB of RAM). Conducting the simulation on a more powerful machine reduced the runtime to around 11 minutes, indicating the possibility for algorithm optimisation.

## 5.3. Future Improvements

Current implementation relies on loops to run all simulations where the data collection happens after each steps, which also require additional steps to do post-processing of the output data. Efforts could be made to improve the algorithm with the following focuses:

- Specify the data collection process so that it only occurs during specific events (e.g., removal of agents).
- Use a more efficient mechanism to execute loops of the model in order to reduce computational load and runtime.
- Improve the `model_run` structure so it can also accept the number of seeds, roads, and scenarios as arguments when executed, enabling a simpler experimental setup.

# References


[1] Marko A. Hofmann. “Criteria for Decomposing Systems Into Components in Modeling and Simulation: Lessons Learned with Military Simulations”. In: *SIMULATION* 80.7-8 (2004), pp. 357–365. DOI: 10.1177/0037549704049876.

[2] Staff Correspondent. *Most of Chittagong port containers move by road, raising costs and delays*. https://en.bonikbarta.com/business/Ks7Xi2nNN22qzB33. Bonik Barta. Accessed: 2026-03-02. Dec. 2025.

[3] UNISCO. *Chittagong, Bangladesh*. https://www.unisco.com/international-ports/chittagong-bangladesh. Accessed: 2026-03-02. n.d.

# A

# Data and Code Availability

All data processing, analysis, and visualization code used in this study are openly available to ensure transparency and reproducibility. The complete codebase, along with documentation and links to the underlying datasets, is hosted in a public GitHub repository at:

`https://github.com/taufik-my/ASIM_Team14/tree/main/EPA133a-G14-A2`

# B

# Use of AI Tools

## B.1. How AI Was Used

- AI (Gemini and ChatGPT) was used to improve writing style and spell check. Grammarly was also used to review our writing. We reviewed the output to ensure consistency with our writing style.
- AI (Gemini and ChatGPT) was used to reformat our writing in LaTeX where needed.
- AI (Gemini, ChatGPT, and Claude) was used to help debug issues in our code.

## B.2. How AI Was Not Used

- AI was not used to generate the final results or interpret them.
- AI was not used to identify errors in the dataset.
- AI was not used to make unverified claims.

# C

# Team Contributions

This assignment was completed collaboratively by all members of Team 14. The table below summarises the primary contributions of each team member. All members participated in discussions, validation of results, and revision of the final report.

**Table C.1:** Overview of team member contributions

| Student Name | Main Contribution |
|---|---|
| Bayu Jamalullael | Data Preparation; cleaning and combining road and bridge dataset to be a model-ready dataframe |
| Brenda Escobar Arriaga | Introduction, collaboration for component modification, model overview description, interpretation and conclusion |
| Brian Parsaoran | Model and model_run, data collector, automated execution, cohesion and modularity |
| Taufik M. Yusup | Data visualization for the results and bonus 1, preparing GitHub repository and LaTeX report |
| Zhafran Sidik | Component architecture and writing |